

Supply Chain Intelligence Analytics Platform

SUPPLY CHAIN

PYTHON

SQL SERVER

POWER BI

LOGISTICS ANALYTICS



180,511

Clean Records

35

Final Columns

\$36.78M

Revenue

\$3.97M

Profit

118

Products

57.28%

Delay Orders

Contents

1. Executive Summary
2. Dataset, Domain Background, and Business Problem
3. Data Dictionary
4. Project Architecture and Analytics Flow
5. Python EDA and Data Cleaning Pipeline
6. Python Cleaning Evidence Screenshots
7. SQL Server Analysis and Insights
8. SQL Server Screenshot Evidence
9. Power BI Dashboard and KPI Reporting
10. Results, Conclusion, and Future Scope

This single version combines the strong written structure from the straight report with the cleaner visual design, dashboard image, Python evidence screenshots, and SQL evidence screenshots from the platform PDF. Duplicate content was avoided and the screenshots were placed after the section where they are most useful.

1. Executive Summary

The Supply Chain Intelligence Analytics Platform is an end-to-end analytics project that transforms raw logistics transactions into actionable operational intelligence. The workflow starts with raw CSV data, uses Python for cleaning and feature engineering, SQL Server for business analysis, and Power BI for executive dashboard reporting.

180,511 Dataset Records	35 Columns After Cleaning	\$36.78M Total Revenue
\$3.97M Total Profit	118 Unique Products	57.28% Delayed Orders %

Project Goal

Build a complete analytical solution that cleans raw supply chain data, produces SQL-based business insights, and visualizes key performance indicators in Power BI for management decision-making.

Tools Used

- Python and Pandas for EDA, data cleaning, datatype conversion, duplicate handling, and feature engineering.
- SQL Server / SSMS for query-based analysis across sales, profit, shipping delays, regions, departments, products, and customer segments.
- Power BI for an executive dashboard with KPI cards, filters, trend charts, product risk analysis, regional comparisons, and key insights.
- PDF reporting for academic documentation and project evidence.

2. Dataset, Domain Background, and Business Problem

About the Dataset

Type	Supply Chain / Retail Logistics Dataset
Nature	Structured tabular dataset
Level	Order and shipment transaction level
Raw Size	180,519 rows x 20 columns
Final Size	180,511 rows x 35 columns after cleaning and feature engineering
Date Range	2015-01-01 to 2018-01-31
Data Mix	Numerical: sales, profit, quantity, shipping days, profit margin. Categorical: market, region, department, shipping mode, product, segment, delivery status.

Domain Background

In supply chain operations, every order moves through product selection, order processing, shipment scheduling, delivery execution, and customer fulfillment. Analytics helps the business identify where delays occur, which products create risk, which markets generate profit, and how delivery performance affects business outcomes.

- Shipping and delivery data helps measure service reliability.
- Sales and profit data helps identify high-performing departments and categories.
- Regional data highlights strong markets and regions requiring logistics improvement.
- Product-level analysis detects inventory, supplier, and fulfillment bottlenecks.

Business Problem

How can a supply chain business understand shipping delays, profitability patterns, product risk, and regional performance to improve operational efficiency and business decision-making?

Problem Breakdown

- Delivery Delay Problem: Which shipping modes, products, and regions experience the highest delay rates?
- Profitability Problem: Which regions and departments generate revenue but struggle with margins?
- Product Risk Problem: Which products create high operational risk due to delayed deliveries?
- Customer Segment Problem: Which customer segments contribute the most revenue and profit?
- Seasonal Trend Problem: Which months generate peak sales and profit performance?
- Executive Monitoring Problem: How can management track KPIs through one Power BI dashboard?

3. Data Dictionary

The following data dictionary consolidates the operational, commercial, and delivery-related fields used in the project. Engineered fields are included because they are important for SQL queries and Power BI visuals.

Column Name	Description
order_id	Unique identifier for each order.
order_date	Date and time when the order was placed.
order_status	Order status such as Complete, Pending, Closed, Canceled, Processing, or Payment Review.
customer_segment	Customer group such as Consumer, Corporate, or Home Office.
category_name	Product category associated with the order, such as Fishing, Cleats, Apparel, or Camping.
department_name	Business department such as Fan Shop, Apparel, Golf, Footwear, Fitness, or Outdoors.
product_name	Specific product purchased in the transaction.
sales	Revenue amount generated by the order.
order_profit_per_order	Profit generated by the order after order-level cost impact.
shipping_mode	Shipping method such as Standard Class, First Class, Second Class, or Same Day.
days_shipping_real	Actual number of days taken for shipping.
days_shipment_scheduled	Scheduled shipping duration.
delivery_status	Delivery result such as Late Delivery, Shipping On Time, Advance Shipping, or Shipping Canceled.
market	Market group such as Europe, LATAM, USCA, Africa, or Pacific Asia.
order_region	Region where the order was delivered.
order_quantity	Quantity ordered by the customer.
shipping_delay_days	Engineered field calculated as actual shipping days minus scheduled shipping days.
delivery_performance	Engineered field classifying shipments as Early, On Time, or Delayed.
is_late_delivery	Binary flag used to identify late deliveries for analysis.
profit_margin_pct	Calculated percentage based on order profit divided by sales.
revenue_per_unit	Revenue per item based on sales and order quantity.
order_size	Engineered order quantity group: Small, Medium, or Large.
profit_type	Profit or Loss label based on order_profit_per_order.
sales_tier	Engineered sales category: Low, Medium, or High.

4. Project Architecture and Analytics Flow

The project follows a full-stack data analytics flow: raw data is cleaned and enriched, stored or analyzed through SQL Server, visualized in Power BI, and documented in a final professional report.

Stage	Tool	Output
Raw Dataset	CSV	180,519 raw transaction records and 20 initial columns.
Python Cleaning	Jupyter Notebook	Cleaned dataset with standardized columns, engineered features, zero duplicates, and zero missing values.
SQL Server	SSMS	16 business-focused analytical queries and output tables.
Power BI Dashboard	Power BI Desktop	Interactive dashboard with KPI cards, slicers, charts, and business insights.
Professional Report	PDF	Merged documentation with content, dashboard image, Python evidence, and SQL evidence.

Raw CSV	Python Cleaning	SQL Server	Power BI	Report
180,519 records	Feature engineering	16 analytical queries	Executive KPIs	Final evidence PDF

End-to-end output: raw data is transformed into cleaned features, SQL business intelligence outputs, and a Power BI dashboard for operational decision-making.

5. Python EDA and Data Cleaning Pipeline

The Python cleaning workflow prepares the raw supply chain dataset for SQL Server analysis and Power BI reporting. The process is arranged as what was done, why it was done, and the outcome produced.

Step	Process	Purpose / Outcome
01	Import & Inspect	Imported pandas and NumPy, loaded supply_chain.csv, checked raw shape, previewed records, and inspected datatypes.
02	Initial Exploration	Reviewed df.info(), df.describe(), missing values, duplicate count, and important categorical values.
03	Standardize Columns	Converted headers into clean lowercase snake_case names suitable for SQL Server and Power BI.
04	Remove Redundant Columns	Detected benefit_per_order as duplicate of order_profit_per_order and removed redundant data.
05	Missing & Duplicate Handling	Handled missing values, removed 8 duplicate rows, and reset the index.
06	Clean Text	Removed extra spaces and standardized categorical values using title case.
07	Date-Time Conversion	Converted order_date to datetime and extracted year, month, day, quarter, and weekday fields.
08	Feature Engineering	Created shipping_delay_days, delivery_performance, is_late_delivery, profit_margin_pct, revenue_per_unit, order_size, profit_type, and sales_tier.
09	Advanced Preprocessing	Standardized country names, created season feature, flagged outliers, and converted repeated categories.
10	Validation & Export	Validated 180,511 rows x 35 columns, zero missing values, zero duplicates, and exported the cleaned CSV.

Final Python Cleaning Outcome

- Final row count: 180,511 records after removing 8 duplicate rows.
- Final column count: 35 columns after cleaning and feature engineering.
- Data quality checks: zero missing values and zero duplicate rows in the final cleaned dataset.
- Core engineered fields: shipping_delay_days, delivery_performance, is_late_delivery, profit_margin_pct, revenue_per_unit, order_size, profit_type, and sales_tier.

6. Python Cleaning Using Pandas & Numpy

The following screenshots document the notebook workflow from data loading to final export. They are arranged chronologically so the evidence follows the actual cleaning pipeline.

Python Step 1: Import required libraries and set display options.

```
[1]: # 1. Import Required Libraries
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", "{:.2f}".format)

print("✅ Libraries imported successfully")

✅ Libraries imported successfully
```

Figure P1: Python cleaning screenshot 1.

Python Step 2: Load supply_chain.csv and preview the raw dataset.

```
# 2. Load Dataset
df = pd.read_csv("supply_chain.csv")

print(f"Dataset Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
df.head()
```

Dataset Shape: 180,519 rows x 20 columns

	Order Id	order date (DateOrders)	Order Status	Customer Segment	Category Name	Department Name	Product Name	Sales	Order Profit Per Order	Benefit per order	Shipping Mode	Days for shipping (real)	Days for shipment (scheduled)	Delivery Status	Late_d
0	77202	1/31/2018 22:56	COMPLETE	Consumer	Sporting Goods	Fitness	Smart watch	327.75	91.25	91.25	Standard Class	3	4	Advance shipping	
1	75939	1/13/2018 12:27	PENDING	Consumer	Sporting Goods	Fitness	Smart watch	327.75	-249.09	-249.09	Standard Class	5	4	Late delivery	
2	75938	1/13/2018 12:06	CLOSED	Consumer	Sporting Goods	Fitness	Smart watch	327.75	-247.78	-247.78	Standard Class	4	4	Shipping on time	
3	75937	1/13/2018 11:45	COMPLETE	Home Office	Sporting Goods	Fitness	Smart watch	327.75	22.86	22.86	Standard Class	3	4	Advance shipping	
4	75936	1/13/2018 11:24	PENDING_PAYMENT	Corporate	Sporting Goods	Fitness	Smart watch	327.75	134.21	134.21	Standard Class	2	4	Advance shipping	

Figure P2: Python cleaning screenshot 2.

Python Step 3: Initial dataset information and datatype review.

```
# 3. Initial Exploration
# Dataset information
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180519 entries, 0 to 180518
Data columns (total 20 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Order Id                             180519 non-null  int64
 1   order date (DateOrders)              180519 non-null  object
 2   Order Status                         180519 non-null  object
 3   Customer Segment                    180519 non-null  object
 4   Category Name                       180519 non-null  object
 5   Department Name                     180519 non-null  object
 6   Product Name                        180519 non-null  object
 7   Sales                               180519 non-null  float64
 8   Order Profit Per Order               180519 non-null  float64
 9   Benefit per order                   180519 non-null  float64
10   Shipping Mode                       180519 non-null  object
11   Days for shipping (real)             180519 non-null  int64
12   Days for shipment (scheduled)       180519 non-null  int64
13   Delivery Status                     180519 non-null  object
14   Late_delivery_risk                  180519 non-null  int64
15   Order Region                       180519 non-null  object
16   Market                             180519 non-null  object
17   Order Country                      180519 non-null  object
18   Order Item Quantity                 180519 non-null  int64
19   Customer State                     180519 non-null  object
dtypes: float64(3), int64(5), object(12)
memory usage: 27.5+ MB
```

Figure P3: Python cleaning screenshot 3.

Python Step 4: Statistical summary for numerical and categorical columns.

```
# Statistical summary for numeric + categorical columns
df.describe(include="all")
```

	Order Id	order date (DateOrders)	Order Status	Customer Segment	Category Name	Department Name	Product Name	Sales	Order Profit Per Order	Benefit per order	Shipping Mode	Days for shipping (real)	Days for shipment (scheduled)	Deliv St
count	180519.00	180519	180519	180519	180519	180519	180519	180519.00	180519.00	180519.00	180519	180519.00	180519.00	180519
unique	NaN	65752	9	3	50	11	118	NaN	NaN	NaN	4	NaN	NaN	NaN
top	NaN	12/14/2016 12:29	COMPLETE	Consumer	Cleats	Fan Shop	Perfect Fitness Perfect Rip Deck	NaN	NaN	NaN	Standard Class	NaN	NaN	deli
freq	NaN	5	59491	93504	24551	66861	24515	NaN	NaN	NaN	107752	NaN	NaN	98
mean	36221.89	NaN	NaN	NaN	NaN	NaN	NaN	203.77	21.97	21.97	NaN	3.50	2.93	NaN
std	31045.28	NaN	NaN	NaN	NaN	NaN	NaN	122.27	104.42	104.42	NaN	1.62	1.27	NaN

Figure P4: Python cleaning screenshot 4.

Python Step 5: Missing value check before cleaning.

```
# Missing values before cleaning
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_df = pd.DataFrame({"Missing Count": missing, "Missing %": missing_pct})
missing_df[missing_df["Missing Count"] > 0]
missing_df.head()
```

	Missing Count	Missing %
Order Id	0	0.00
order date (DateOrders)	0	0.00
Order Status	0	0.00
Customer Segment	0	0.00
Category Name	0	0.00

Figure P5: Python cleaning screenshot 5.

Python Step 6: Duplicate row count before cleaning.

```
# Duplicate rows before cleaning
print("Duplicate Rows:", df.duplicated().sum())
```

Duplicate Rows: 8

Figure P6: Python cleaning screenshot 6.

Python Step 7: Unique values in important categorical columns.

```
!]: # Unique values in important categorical columns
cat_cols = ["Order Status", "Customer Segment", "Shipping Mode", "Delivery Status", "Market", "Department Name"]

for col in cat_cols:
    if col in df.columns:
        print(f"\n{col}:")
        print(df[col].unique())

Order Status:
['COMPLETE' 'PENDING' 'CLOSED' 'PENDING_PAYMENT' 'CANCELED' 'PROCESSING'
 'SUSPECTED_FRAUD' 'ON_HOLD' 'PAYMENT_REVIEW']

Customer Segment:
['Consumer' 'Home Office' 'Corporate']

Shipping Mode:
['Standard Class' 'First Class' 'Second Class' 'Same Day']

Delivery Status:
['Advance shipping' 'Late delivery' 'Shipping on time' 'Shipping canceled']

Market:
['Pacific Asia' 'USCA' 'Africa' 'Europe' 'LATAM']

Department Name:
```

Figure P7: Python cleaning screenshot 7.

Python Step 8: Original column names before standardization.

```
# 4. Standardize Column Names
print("Original Columns:")
print(df.columns.tolist())

Original Columns:
['Order Id', 'order date (DateOrders)', 'Order Status', 'Customer Segment', 'Category Name', 'Department Name', 'Product Name', 'Sales', 'Order Profit Per Order', 'Benefit per order', 'Shipping Mode', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Delivery Status', 'Late_delivery_risk', 'Order Region', 'Market', 'Order Country', 'Order Item Quantity', 'Customer State']
```

Figure P8: Python cleaning screenshot 8.

Python Step 9: Standardized column names and selected renaming.

```
# Standardize column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(r"[\s]+", "_", regex=True)
    .str.replace(r"[^a-z0-9_]", "", regex=True)
    .str.strip("_")
)

# Rename selected columns for better readability
df.rename(columns={
    "order_date_dateorders": "order_date",
    "days_for_shipping_real": "days_shipping_real",
    "days_for_shipment_scheduled": "days_shipment_scheduled",
    "order_profit": "order_profit",
    "order_item_quantity": "order_quantity"
}, inplace=True)

print("✅ Standardized Columns:")
print(df.columns.tolist())

✅ Standardized Columns:
['order_id', 'order_date', 'order_status', 'customer_segment', 'category_name', 'department_name', 'product_name', 'sales', 'order_profit_per_order',
'benefit_per_order', 'shipping_mode', 'days_shipping_real', 'days_shipment_scheduled', 'delivery_status', 'late_delivery_risk', 'order_region', 'marke
t', 'order_country', 'order_quantity', 'customer_state']
```

Figure P9: Python cleaning screenshot 9.

Python Step 10: Removed redundant benefit_per_order column.

```
# 5. Remove Redundant Columns
# Check if benefit_per_order and order_profit_per_order contain the same values

if {"benefit_per_order", "order_profit_per_order"}.issubset(df.columns):
    same_values = np.allclose(
        df["benefit_per_order"].fillna(0),
        df["order_profit_per_order"].fillna(0)
    )

    print("benefit_per_order same as order_profit_per_order:", same_values)

    if same_values:
        df.drop(columns=["benefit_per_order"], inplace=True)
        print("✅ Removed redundant column: benefit_per_order")

print("Current Shape:", df.shape)

benefit_per_order same as order_profit_per_order: True
✅ Removed redundant column: benefit_per_order
Current Shape: (180519, 19)
```

Figure P10: Python cleaning screenshot 10.

Python Step 11: Removed fully empty rows.

```
# 6. Handle Missing and Duplicate Values

# Remove fully empty rows
before_rows = df.shape[0]
df.dropna(how="all", inplace=True)
print(f"Removed fully empty rows: {before_rows - df.shape[0]}")
```

Removed fully empty rows: 0

Figure P11: Python cleaning screenshot 11.

Python Step 12: Handled missing values for numerical and categorical columns.

```
# Fill numerical missing values with median
num_cols = df.select_dtypes(include=["int64", "float64"]).columns

for col in num_cols:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())

# Fill categorical missing values with mode
cat_cols = df.select_dtypes(include=["object"]).columns

for col in cat_cols:
    if df[col].isnull().sum() > 0:
        mode_value = df[col].mode(dropna=True)
        df[col] = df[col].fillna(mode_value.iloc[0] if not mode_value.empty else "Unknown")

print("✅ Missing values handled")
print(df.isnull().sum())
```

```
✅ Missing values handled
order_id          0
order_date        0
order_status      0
customer_segment  0
category_name     0
department_name   0
product_name      0
sales             0
order_profit_per_order  0
shipping_mode     0
days_shipping_real  0
days_shipment_scheduled  0
delivery_status   0
late_delivery_risk  0
order_region      0
market            0
order_country     0
```

Figure P12: Python cleaning screenshot 12.

Python Step 13: Removed duplicate rows and reset index.

```
# Remove duplicate rows
before = df.shape[0]
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)
after = df.shape[0]

print(f"✅ Removed {before - after} duplicate rows")
print("Current Shape:", df.shape)
```

✅ Removed 8 duplicate rows
Current Shape: (180511, 19)

Figure P13: Python cleaning screenshot 13.

Python Step 14: Cleaned text columns and standardized categories.

```
# 7. Clean Text and Categorical Columns

# Strip extra spaces from text columns
str_cols = df.select_dtypes(include="object").columns

for col in str_cols:
    df[col] = df[col].astype(str).str.strip()

print("✅ Extra spaces removed from text columns")

✅ Extra spaces removed from text columns

# Standardize selected categorical columns to Title Case
cat_cols = [
    "order_status", "customer_segment", "shipping_mode", "delivery_status",
    "market", "department_name", "order_region", "order_country",
    "customer_state", "category_name"
]

for col in cat_cols:
    if col in df.columns:
        df[col] = df[col].str.title()

print("✅ Categorical text standardized")

✅ Categorical text standardized
```

Figure P14: Python cleaning screenshot 14.

Python Step 15: Converted order_date into datetime format.

```
# 8. Date-Time Conversion

# Check raw date values
print(df["order_date"].head())
print("Current dtype:", df["order_date"].dtype)

0    1/31/2018 22:56
1    1/13/2018 12:27
2    1/13/2018 12:06
3    1/13/2018 11:45
4    1/13/2018 11:24
Name: order_date, dtype: object
Current dtype: object

# Convert order_date to datetime
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")

print("New dtype:", df["order_date"].dtype)
print("Date Range:", df["order_date"].min(), "to", df["order_date"].max())

New dtype: datetime64[ns]
Date Range: 2015-01-01 00:00:00 to 2018-01-31 23:38:00
```

Figure P15: Python cleaning screenshot 15.

Python Step 16: Extracted useful date features from order_date.

```
# Extract useful date features
df["order_year"] = df["order_date"].dt.year
df["order_month"] = df["order_date"].dt.month
df["order_day"] = df["order_date"].dt.day
df["order_quarter"] = df["order_date"].dt.quarter
df["order_weekday"] = df["order_date"].dt.day_name()

df[["order_date", "order_year", "order_month", "order_day", "order_quarter", "order_weekday"]].head()
```

	order_date	order_year	order_month	order_day	order_quarter	order_weekday
0	2018-01-31 22:56:00	2018	1	31	1	Wednesday
1	2018-01-13 12:27:00	2018	1	13	1	Saturday
2	2018-01-13 12:06:00	2018	1	13	1	Saturday
3	2018-01-13 11:45:00	2018	1	13	1	Saturday
4	2018-01-13 11:24:00	2018	1	13	1	Saturday

Figure P16: Python cleaning screenshot 16.

Python Step 17: Created shipping delay and delivery performance features.

```
# 9. Feature Engineering

# Shipping Delay = Actual shipping days - Scheduled shipping days
df["shipping_delay_days"] = df["days_shipping_real"] - df["days_shipment_scheduled"]

df["delivery_performance"] = np.where(
    df["shipping_delay_days"] > 0,
    "Delayed",
    np.where(df["shipping_delay_days"] < 0, "Early", "On Time")
)

print(df[["days_shipping_real", "days_shipment_scheduled", "shipping_delay_days", "delivery_performance"]].head())
```

	days_shipping_real	days_shipment_scheduled	shipping_delay_days	delivery_performance
0	3	4	-1	Early
1	5	4	1	Delayed
2	4	4	0	On Time
3	3	4	-1	Early
4	2	4	-2	Early

Figure P17: Python cleaning screenshot 17.

Python Step 18: Created late delivery flag and profit margin percentage.

```
# Late Delivery Flag
df["is_late_delivery"] = (df["delivery_status"].str.lower() == "late delivery").astype(int)

print(df["is_late_delivery"].value_counts())
print(f"Late Delivery Rate: {df['is_late_delivery'].mean() * 100:.2f}%")
```

```
is_late_delivery
1    98974
0    81537
Name: count, dtype: int64
Late Delivery Rate: 54.83%
```

```
# Profit Margin %
df["profit_margin_pct"] = np.where(
    df["sales"] != 0,
    (df["order_profit_per_order"] / df["sales"]) * 100,
    np.nan
).round(2)

df["profit_margin_pct"].describe()
```

```
count    180511.00
mean         10.83
std         42.06
min        -275.00
25%          6.22
50%         24.25
75%         33.60
max          50.04
Name: profit_margin_pct, dtype: float64
```

Figure P18: Python cleaning screenshot 18.

Python Step 19: Created revenue per unit and order size category.

```
# Revenue Per Unit
df["revenue_per_unit"] = np.where(
    df["order_quantity"] != 0,
    df["sales"] / df["order_quantity"],
    np.nan
).round(2)

df["revenue_per_unit"].describe()
```

```
count    180511.00
mean       141.23
std        139.73
min         9.99
25%        50.00
50%        59.99
75%       199.99
max       1999.99
Name: revenue_per_unit, dtype: float64
```

```
# Order Size Category
df["order_size"] = pd.cut(
    df["order_quantity"],
    bins=[0, 1, 3, np.inf],
    labels=["Small", "Medium", "Large"],
    include_lowest=True
)

df["order_size"].value_counts()
```

```
order_size
Small      99130
Large      40718
Medium     40663
Name: count, dtype: int64
```

Figure P19: Python cleaning screenshot 19.

Python Step 20: Created profit type and sales tier features.

```
# Profit Type
df["profit_type"] = np.where(df["order_profit_per_order"] >= 0, "Profit", "Loss")

df["profit_type"].value_counts()
```

```
profit_type
Profit    146727
Loss      33784
Name: count, dtype: int64
```

```
# Sales Tier
df["sales_tier"] = pd.cut(
    df["sales"],
    bins=[-np.inf, 100, 300, np.inf],
    labels=["Low", "Medium", "High"]
)

df["sales_tier"].value_counts()
```

```
sales_tier
Medium    115378
Low       39195
High      25938
Name: count, dtype: int64
```

Figure P20: Python cleaning screenshot 20.

Python Step 21: Advanced preprocessing - country name standardization.

```
[27]: # 10. Advanced Feature Engineering & Preprocessing

# 1. Country Name Standardization

country_map = {
    "Estados Unidos": "United States",
    "Alemania": "Germany",
    "Francia": "France",
    "Reino Unido": "United Kingdom",
    "Japón": "Japan",
    "Corea Del Sur": "South Korea",
    "Corea del Sur": "South Korea",
    "Australia": "Australia",
    "Canadá": "Canada",
    "Canada": "Canada",
    "Brasil": "Brazil",
    "México": "Mexico",
    "Italia": "Italy",
    "España": "Spain",
    "Países Bajos": "Netherlands",
    "China": "China",
    "India": "India"
}

if "order_country" in df.columns:
    df["order_country_en"] = df["order_country"].map(country_map).fillna(df["order_country"])
    print("Country name standardization completed.")
    display(df[["order_country", "order_country_en"]].head())
else:
    print("Column 'order_country' not found.")

Country name standardization completed.
```

	order_country	order_country_en
0	Indonesia	Indonesia
1	India	India
2	India	India
3	Australia	Australia
4	Australia	Australia

Figure P21: Python cleaning screenshot 21.

Python Step 22: Created season feature and sales outlier flag.

```
# 2. Season Feature Creation

if "order_month" in df.columns:
    df["order_season"] = df["order_month"].map({
        12: "Winter", 1: "Winter", 2: "Winter",
        3: "Spring", 4: "Spring", 5: "Spring",
        6: "Summer", 7: "Summer", 8: "Summer",
        9: "Fall", 10: "Fall", 11: "Fall"
    })
    print("Season feature created successfully.")
    display(df[["order_month", "order_season"]].head())
else:
    print("Column 'order_month' not found. Create order_month from order date first.")

Season feature created successfully.
```

	order_month	order_season
0	1	Winter
1	1	Winter
2	1	Winter
3	1	Winter
4	1	Winter

```
# 3. Outlier Flag Creation Instead of Removing Outliers

if "sales" in df.columns:
    Q1 = df["sales"].quantile(0.25)
    Q3 = df["sales"].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df["is_sales_outlier"] = ((df["sales"] < lower) | (df["sales"] > upper)).astype(int)

    print("Sales outlier flag created successfully.")
    print("Lower Limit:", lower)
    print("Upper Limit:", upper)
    print(df["is_sales_outlier"].value_counts())
else:
    print("Column 'sales' not found.")

Sales outlier flag created successfully.
Lower Limit: -149.97500900000007
Upper Limit: 569.9050254000001
is_sales_outlier
0    100023
1         488
Name: count, dtype: int64
```

Figure P22: Python cleaning screenshot 22.

Python Step 23: Negative value and business loss validation.

```
# 11. Negative Value and Outlier Detection

# Negative value checks
print("Negative Sales:", (df["sales"] < 0).sum())
print("Negative Order Profit:", (df["order_profit_per_order"] < 0).sum(), "(valid business losses, keep them)")

Negative Sales: 0
Negative Order Profit: 33784 (valid business losses, keep them)
```

Figure P23: Python cleaning screenshot 23.

Python Step 24: Outlier detection summary using IQR method.

```
# Outlier detection using IQR method
numeric_cols = df.select_dtypes(include=["int64", "float64"]).columns

outlier_summary = []

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    count = ((df[col] < lower) | (df[col] > upper)).sum()
    outlier_summary.append([col, count, lower, upper])

outlier_df = pd.DataFrame(outlier_summary, columns=["Column", "Outlier Count", "Lower Limit", "Upper Limit"])
outlier_df
```

	Column	Outlier Count	Lower Limit	Upper Limit
0	order_id	0	-36071.00	108273.00
1	sales	488	-149.98	569.91
2	order_profit_per_order	18942	-79.70	151.50
3	days_shipping_real	0	-2.50	9.50
4	days_shipment_scheduled	0	-1.00	7.00
5	late_delivery_risk	0	-1.50	2.50
6	order_quantity	0	-2.00	6.00
7	shipping_delay_days	35701	-1.50	2.50
8	is_late_delivery	0	-1.50	2.50
9	profit_margin_pct	16825	-34.85	74.67
10	revenue_per_unit	2048	-174.99	424.98
11	is_sales_outlier	488	0.00	0.00

Figure P24: Python cleaning screenshot 24.

Python Step 25: Final cleaned dataset validation summary.

```
# 12. Final validation

print("=" * 60)
print("FINAL CLEANED DATASET SUMMARY")
print("=" * 60)
print(f"Total Rows      : {df.shape[0]:,}")
print(f"Total Columns    : {df.shape[1]}")
print(f"Missing Values    : {df.isnull().sum().sum()}")
print(f"Duplicate Rows    : {df.duplicated().sum()}")
print(f>Date Range      : {df['order_date'].min()} to {df['order_date'].max()}")
print("=" * 60)

=====
FINAL CLEANED DATASET SUMMARY
=====
Total Rows      : 180,511
Total Columns   : 35
Missing Values  : 0
Duplicate Rows  : 0
Date Range     : 2015-01-01 00:00:00 to 2018-01-31 23:38:00
=====
```

Figure P25: Python cleaning screenshot 25.

Python Step 26: Final datatype verification.

```
# Final data types
df.dtypes

order_id                int64
order_date              datetime64[ns]
order_status            object
customer_segment        object
category_name           object
department_name         object
product_name            object
sales                  float64
order_profit_per_order  float64
shipping_mode           object
days_shipping_real      int64
days_shipment_scheduled int64
delivery_status         object
late_delivery_risk       int64
order_region            object
market                  object
order_country           object
order_quantity          int64
customer_state          object
order_year              int32
order_month             int32
order_day               int32
order_quarter           int32
order_weekday           object
shipping_delay_days      int64
delivery_performance     object
is_late_delivery         int64
profit_margin_pct       float64
revenue_per_unit        float64
order_size              category
profit_type             object
sales_tier              category
order_country_en        object
order_season            object
is_sales_outlier        int64
dtype: object
```

Figure P26: Python cleaning screenshot 26.

Python Step 27: Converted repeated categorical columns to category datatype.

```
# Convert repeated categorical columns to category datatype
category_cols = [
    "order_status",
    "customer_segment",
    "category_name",
    "department_name",
    "shipping_mode",
    "delivery_status",
    "order_region",
    "market",
    "order_country",
    "customer_state",
    "delivery_performance",
    "order_weekday",
    "order_size",
    "profit_type",
    "sales_tier",
    "order_country_en",
    "order_season"
]

for col in category_cols:
    if col in df.columns:
        df[col] = df[col].astype("category")

print("✅ Categorical columns converted to category datatype")
✅ Categorical columns converted to category datatype
```

Figure P27: Python cleaning screenshot 27.

Python Step 28: Final preview and cleaned dataset export.

```
# Final preview
df.head()
```

	order_id	order_date	order_status	customer_segment	category_name	department_name	product_name	sales	order_profit_per_order	shipping_mode	days_sl
0	77202	2018-01-31 22:56:00	Complete	Consumer	Sporting Goods	Fitness	Smart watch	327.75	91.25	Standard Class	
1	75939	2018-01-13 12:27:00	Pending	Consumer	Sporting Goods	Fitness	Smart watch	327.75	-249.09	Standard Class	
2	75938	2018-01-13 12:06:00	Closed	Consumer	Sporting Goods	Fitness	Smart watch	327.75	-247.78	Standard Class	
3	75937	2018-01-13 11:45:00	Complete	Home Office	Sporting Goods	Fitness	Smart watch	327.75	22.86	Standard Class	
4	75936	2018-01-13 11:24:00	Pending_Payment	Corporate	Sporting Goods	Fitness	Smart watch	327.75	134.21	Standard Class	

```
# 13. Export Cleaned Dataset

df.to_csv("supply_chain_cleaned.csv", index=False, encoding="utf-8")

print("✅ Cleaned dataset exported successfully: supply_chain_cleaned.csv")
print(f"Final Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")

✅ Cleaned dataset exported successfully: supply_chain_cleaned.csv
Final Shape: 180,511 rows x 35 columns
```

Figure P28: Python cleaning screenshot 28.

7. SQL Server Analysis and Insights

SQL Server was used after Python cleaning to generate structured business insights. The analysis focused on revenue, profit, delivery delays, customer segments, operational efficiency, product risk, and department performance.

Query Area	Main Insight
Sales & Profit	Total sales reached \$36.78M and total profit reached \$3.97M.
Shipping Mode Risk	Standard Class generated the largest late-delivery volume; First Class also showed serious late-delivery exposure.
Delay Pattern	+1 day delay was the most common operational delay pattern, showing frequent minor dispatch or warehouse delays.
Category Performance	Fishing, Cleats, Camping & Hiking, Cardio Equipment, and Women's Apparel were leading categories by revenue.
Region Performance	Europe and LATAM dominated revenue contribution; Africa showed strong average profit margin.
Customer Segment	Consumer segment generated the highest revenue and order count.
Loss Products	SOLE E35 Elliptical, SOLE E25 Elliptical, and Bushnell Pro X7 showed losses despite sales.
High-Risk Products	Ogio Race Golf Shoes and Team Golf St. Louis Cardinals Putter Grip showed high delay risk above 65%.
Order Size	Large orders generated the highest average sales and profit.

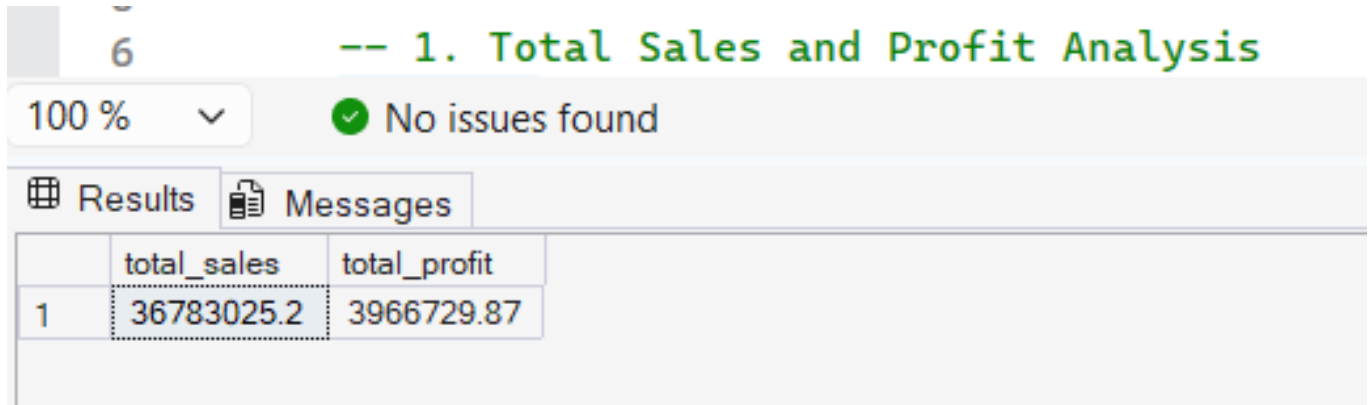
Key SQL Findings

- Total revenue reached \$36.78M and total profit reached \$3.97M.
- Standard Class produced the largest volume of late deliveries, while First Class also showed serious exposure.
- The most common delay pattern was +1 day, suggesting many delays are small but frequent operational issues.
- Fan Shop led revenue and profit, while selected products showed high delay risk above 65%.
- Europe and LATAM dominated revenue contribution, and Africa showed strong operational efficiency by average margin.
- Delayed deliveries had lower average profit than on-time and early shipments, suggesting delivery issues carry operational cost impact.

8. SQL Server For Analysis

The SQL screenshots are placed after the SQL insight section because they directly support the query-based findings. They are arranged from total sales and profit through shipping, region, customer, product, and order-size analysis.

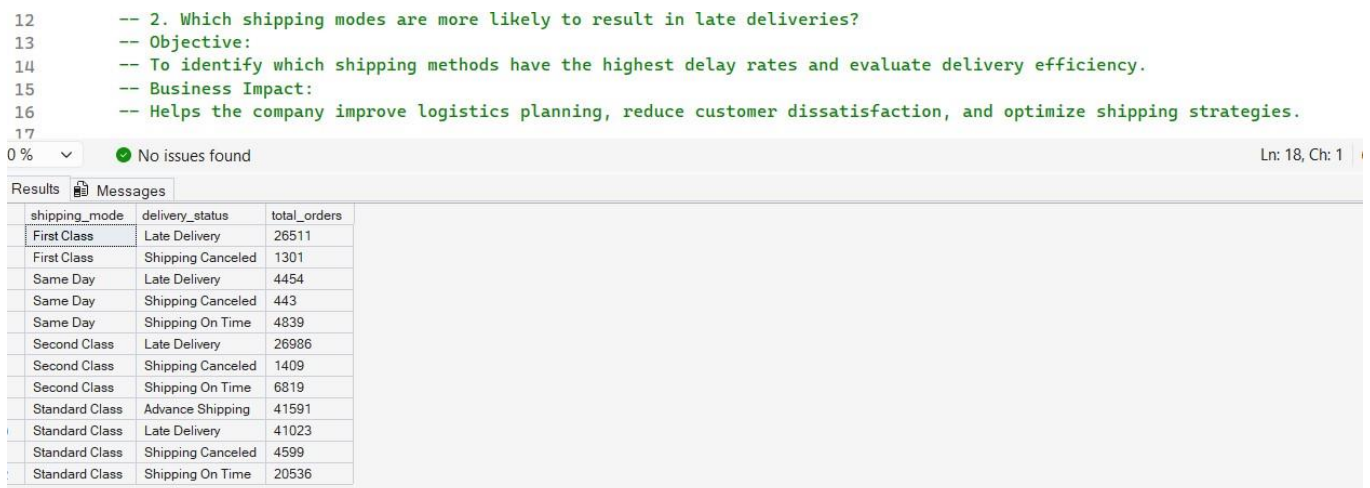
SQL Query 1: Total Sales and Profit Analysis.



	total_sales	total_profit
1	36783025.2	3966729.87

Figure S1: SQL Server output screenshot 1.

SQL Query 2: Shipping modes more likely to result in late deliveries.



shipping_mode	delivery_status	total_orders
First Class	Late Delivery	26511
First Class	Shipping Canceled	1301
Same Day	Late Delivery	4454
Same Day	Shipping Canceled	443
Same Day	Shipping On Time	4839
Second Class	Late Delivery	26986
Second Class	Shipping Canceled	1409
Second Class	Shipping On Time	6819
Standard Class	Advance Shipping	41591
Standard Class	Late Delivery	41023
Standard Class	Shipping Canceled	4599
Standard Class	Shipping On Time	20536

Figure S2: SQL Server output screenshot 2.

SQL Query 3: Shipping delay frequency and operational consistency pattern.

```

25
26      -- 3. How frequently do shipping delays occur, and what do their patterns tell us about operational consistency?
27      -- Objective:
28      -- To analyze how often delays happen and detect recurring operational issues in the supply chain process.
29      -- Business Impact:
30      -- Helps management understand operational stability and improve delivery performance consistency.
31

```

% ✔ No issues found Ln: 32, Ch: 1

Results

shipping_delay_days	total_orders	percentage_of_orders
4	6983	3.87000000000000
3	7052	3.91000000000000
2	28717	15.91000000000000
1	60644	33.60000000000000
0	33750	18.70000000000000
-1	21699	12.02000000000000
-2	21666	12.00000000000000

Figure S3: SQL Server output screenshot 3.

SQL Query 4: Regions and time periods where delays are more common.

```

44      -- 4. Are there specific regions or time periods where shipping delays are more common?
45      -- Objective:
46      -- To identify geographical areas or seasonal periods with higher shipping delays.
47      -- Business Impact:
48      -- Supports region-specific logistics optimization, workforce planning, and seasonal demand management.
49

```

100 % ✔ No issues found

Results

	order_region	order_year	order_month	total_orders	delayed_orders	delay_percentage
1	Southern Europe	2016	8	3	3	100.00000000000000
2	Central Asia	2016	8	4	4	100.00000000000000
3	Western Europe	2016	9	25	24	96.00000000000000
4	Northern Europe	2016	10	40	35	87.50000000000000
5	Eastern Asia	2016	11	22	19	86.36000000000000
6	Southern Europe	2016	9	46	39	84.78000000000000
7	Eastern Asia	2016	10	18	15	83.33000000000000
8	Northern Europe	2016	11	11	9	81.82000000000000
9	Central Asia	2016	12	79	62	78.48000000000000
10	Southern Europe	2017	1	33	24	72.73000000000000
11	Northern Europe	2016	12	32	23	71.88000000000000
12	Southern Europe	2015	5	14	10	71.43000000000000
13	Central Asia	2017	1	71	48	67.61000000000000
14	Southern Africa	2016	12	255	170	66.67000000000000
15	West Africa	2016	8	126	82	65.08000000000000
16	Central Asia	2016	11	123	80	65.04000000000000
17	East Africa	2016	12	450	292	64.89000000000000

Figure S4: SQL Server output screenshot 4.

SQL Query 5: Product categories contributing most to sales and profit.

```

71
72      -- 5. Which product categories contribute the most to overall sales and profit?
73      -- Objective:
74      -- To identify high-performing product categories based on revenue and profitability.
75      -- Business Impact:
76      -- Helps businesses focus on profitable categories and optimize inventory investment.

```

100 % ☐ No issues found

Results Messages

	category_name	total_sales	total_profit
1	Fishing	6929253.71	756220.77
2	Cleats	4431282.89	494595.96
3	Camping & Hiking	4118425.57	427455.57
4	Cardio Equipment	3694843.2	383011.1
5	Women'S Apparel	3147800	350421.03
6	Water Sports	3113444.7	325074.96
7	Indoor/Outdoor Games	2888993.91	318451.43
8	Men'S Footwear	2891627.67	311851.34
9	Shop By Sport	1309402.07	129805.3
10	Computers	663000	69656.81
11	Electronics	371034.64	40891.38
12	Garden	257768.73	33443.01
13	Cameras	267607.69	30289.8
14	Children'S Clothing	232829.2	27178.1
15	Crafts	223356.33	25531.17
16	Women'S Clothing	140283	19102.85
17	Girls' Apparel	151706.2	17288.57

Figure S5: SQL Server output screenshot 5.

SQL Query 6: Regions generating high revenue but lower profitability.

```

85
86      -- 6. Which regions generate the highest revenue but suffer from low profitability?
87      -- Objective:
88      -- To compare sales performance with profitability across regions.
89      -- Business Impact:
90      -- Helps identify regions with high operational costs or pricing inefficiencies.

```

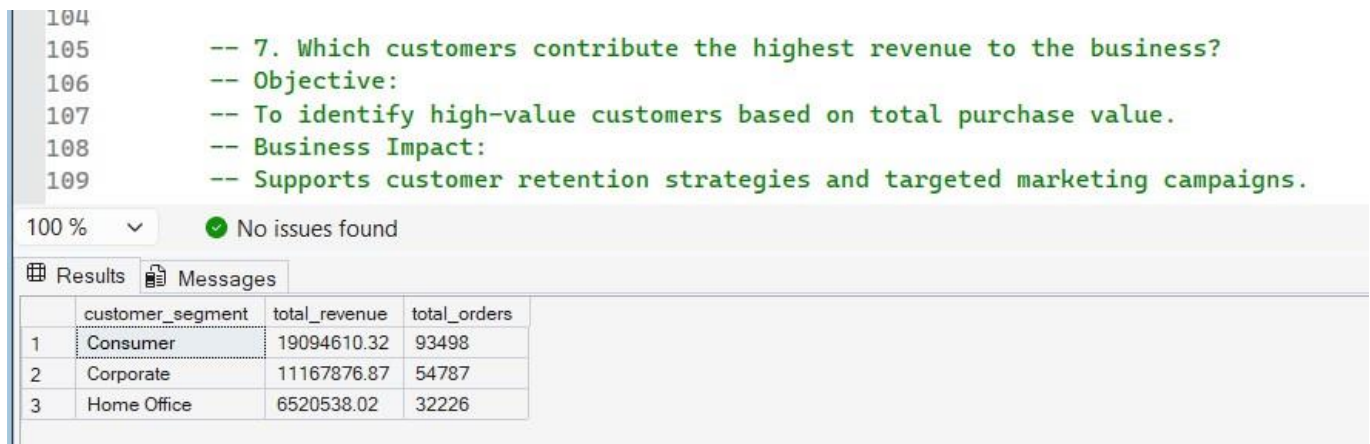
100 % ☐ No issues found

Results Messages

	order_region	total_sales	total_profit	profit_margin_pct
1	Western Europe	5894080.82	625423.12	10.61
2	Central America	5664982.14	616290.09	10.88
3	South America	2960881.41	335154.4	11.32
4	Northern Europe	2155830.65	233450.6	10.83
5	Southern Europe	2047918.82	230829.23	11.27
6	Oceania	2016654.2	201478.02	9.99
7	Southeast Asia	1932495.57	211342.82	10.94
8	Caribbean	1650779.37	171825.64	10.41
9	West Of Usa	1571415.96	164940.66	10.5
10	South Asia	1553560.95	165695.24	10.67
11	Eastern Asia	1486401.34	147368.01	9.91
12	East Of Usa	1370992.01	156245.3	11.4
13	West Asia	1174671.78	118815.41	10.11
14	Us Center	1151155.78	131022.23	11.38
15	South Of Usa	785783.95	88114.88	11.21
16	Eastern Europe	774266.57	79717.05	10.3
17	West Africa	727951.19	80030.27	10.99

Figure S6: SQL Server output screenshot 6.

SQL Query 7: Customers contributing the highest revenue.



```
104
105      -- 7. Which customers contribute the highest revenue to the business?
106      -- Objective:
107      -- To identify high-value customers based on total purchase value.
108      -- Business Impact:
109      -- Supports customer retention strategies and targeted marketing campaigns.
```

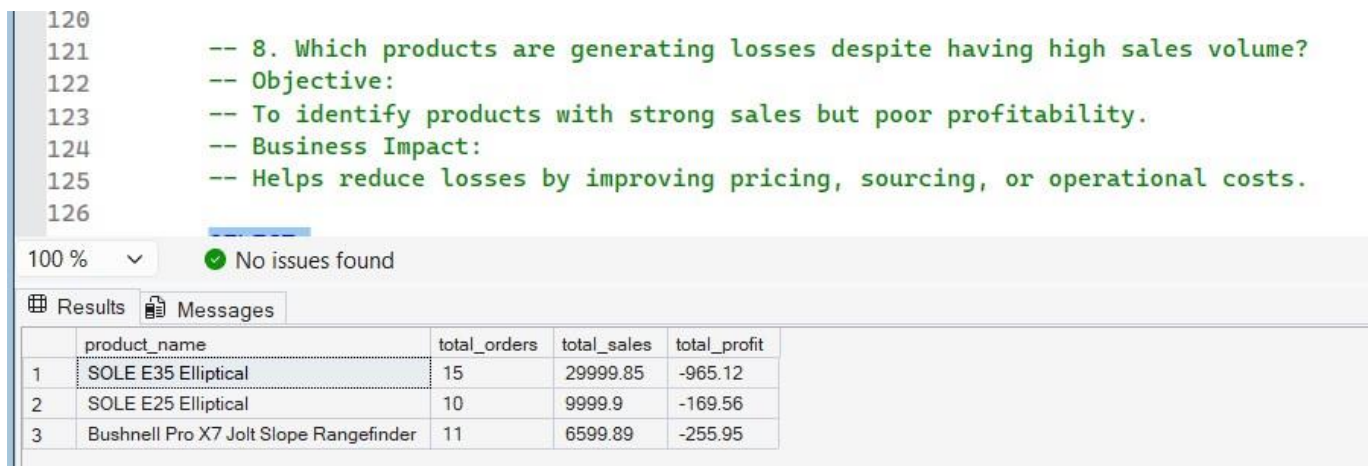
100 % No issues found

Results Messages

	customer_segment	total_revenue	total_orders
1	Consumer	19094610.32	93498
2	Corporate	11167876.87	54787
3	Home Office	6520538.02	32226

Figure S7: SQL Server output screenshot 7.

SQL Query 8: Products generating losses despite high sales volume.



```
120
121      -- 8. Which products are generating losses despite having high sales volume?
122      -- Objective:
123      -- To identify products with strong sales but poor profitability.
124      -- Business Impact:
125      -- Helps reduce losses by improving pricing, sourcing, or operational costs.
126
```

100 % No issues found

Results Messages

	product_name	total_orders	total_sales	total_profit
1	SOLE E35 Elliptical	15	29999.85	-965.12
2	SOLE E25 Elliptical	10	9999.9	-169.56
3	Bushnell Pro X7 Jolt Slope Rangefinder	11	6599.89	-255.95

Figure S8: SQL Server output screenshot 8.

SQL Query 9: Customer segment influence on sales and profitability.

```

137
138      -- 9. How does customer segment influence sales and profitability?
139      -- Objective:
140      -- To analyze purchasing behavior and profitability across customer segments.
141      -- Business Impact:
142      -- Helps businesses design targeted strategies for different customer groups.

```

100 % ✓ No issues found

Results Messages

	customer_segment	total_sales	total_profit	total_orders
1	Consumer	19094610.32	2073366.05	93498
2	Corporate	11167876.87	1202523.48	54787
3	Home Office	6520538.02	690840.34	32226

Figure S9: SQL Server output screenshot 9.

SQL Query 10: Months experiencing peak sales and profit performance.

```

154      -- 10. Which months experience peak sales and profit performance?
155      -- Objective:
156      -- To identify seasonal sales trends and high-performing periods.
157      -- Business Impact:
158      -- Supports demand forecasting and seasonal inventory planning.
159

```

100 % ✓ No issues found

Results Messages

	order_year	order_month	total_sales	total_profit
1	2017	9	1143775.11	122462.39
2	2017	8	1109337.17	131501.16
3	2017	5	1105485.32	115014.64
4	2017	7	1104073.41	113003.74
5	2017	10	1073994.17	113447.17
6	2015	12	1057720.91	110632.89
7	2015	1	1051590.08	111660.74
8	2015	3	1051053.7	113778.21
9	2015	5	1050478.44	112147.9
10	2015	10	1049154.27	101757.87
11	2016	10	1048130.55	116781.97
12	2016	8	1048080.27	110711.54
13	2017	3	1048004.78	118688.9
14	2016	11	1047590.14	109539.2
15	2016	1	1046308.25	106780.95
16	2016	7	1045715.62	119941.72
17	2017	4	1038321.62	115961.06

Figure S10: SQL Server output screenshot 10.

SQL Query 11: Relationship between shipping delays and profitability.

```
169      -- 11. Is there a relationship between shipping delays and profitability?
170      -- Objective:
171      -- To analyze whether delayed deliveries negatively affect business profits.
172      -- Business Impact:
173      -- Helps evaluate the financial impact of operational inefficiencies.
```

00 %

No issues found

Results Messages

delivery_performance	total_orders	avg_profit	avg_delay
On Time	33750	22.53	0
Early	43365	22.47	-1
Delayed	103396	21.59	1

Figure S11: SQL Server output screenshot 11.

SQL Query 12: Markets with the highest operational efficiency.

```
186      -- 12. Which markets have the highest operational efficiency?
187      -- Objective:
188      -- To evaluate market performance based on delivery speed and profitability.
189      -- Business Impact:
190      -- Helps management identify best-performing markets for expansion.
191
```

100 %

No issues found

Results Messages

	market	avg_delivery_days	total_profit	avg_profit_margin
1	Africa	3	252071.18	11.23
2	Europe	3	1169420	11.01
3	Usca	3	564223.78	10.93
4	Latam	3	1123270.13	10.85
5	Pacific Asia	3	857744.78	10.42

Figure S12: SQL Server output screenshot 12.

SQL Query 13: Cities and states with highest order volume and revenue.

201
202 -- 13. Which cities experience the highest order volume and revenue?
203 -- Objective:
204 -- To identify major business hubs based on order activity and revenue generation.
205 -- Business Impact:
206 -- Supports regional expansion and warehouse planning decisions.

100 % No issues found

Results Messages

	customer_state	total_orders	total_sales
1	Pr	69369	14149491.87
2	Ca	29223	5929034.02
3	Ny	11327	2301325.15
4	Tx	9103	1869746.1
5	Il	7630	1561404.57
6	Fl	5456	1110758.41
7	Oh	4095	833180.86
8	Mi	3803	782201.33
9	Pa	3824	772199.01
10	Nj	3191	654152.54

Figure S13: SQL Server output screenshot 13.

SQL Query 14: Departments with highest sales but low profit margins.

216
217 -- 14. Which departments have the highest sales but low profit margins?
218 -- Objective:
219 -- To identify departments generating strong revenue but struggling with profitability.
220 -- Business Impact:
221 -- Helps management improve pricing strategies, reduce operational costs, and increase profit margins.

100 % No issues found

Results Messages

	department_name	total_sales	total_profit	profit_margin_pct
1	Fan Shop	17113070.98	1834083.44	10.72
2	Apparel	7975465.46	881790.49	11.06
3	Golf	4608908.27	497514.9	10.79
4	Footwear	4006498.77	410222.5	10.24
5	Outdoors	1253351.45	145251.46	11.59
6	Technology	1039598.97	113170.01	10.89
7	Fitness	397050.89	46538.06	11.72
8	Discs Shop	228887.73	24193.12	10.57
9	Health And Beauty	106080.48	9493.63	8.95
10	Pet Shop	41524.8	3589.26	8.64
11	Book Shop	12587.4	883.01	7.02

Figure S14: SQL Server output screenshot 14.

SQL Query 15: Products with highest average shipping delays.

```

236      -- 15. Which products have the highest average shipping delays?
237      -- Objective:
238      -- To identify products that frequently experience delayed deliveries.
239      -- Business Impact:
240      -- Helps businesses improve inventory planning, supplier coordination, and logistics management.
241

```

100 % ☒ No issues found

Results Messages

	product_name	total_orders	delayed_orders	delay_percentage
1	Ogio Race Golf Shoes	61	42	68.85000000000000
2	Team Golf St. Louis Cardinals Putter Grip	313	204	65.18000000000000
3	Titleist Small Wheeled Travel Cover	54	35	64.81000000000000
4	Hirzl Women's Sofft Flex Golf Glove	298	190	63.76000000000000
5	adidas Men's Germany Black Crest Away Tee	289	184	63.67000000000000
6	Garmin Approach S3 Golf GPS Watch	77	49	63.64000000000000
7	Under Armour Women's Ignite PIP VI Slide	285	181	63.51000000000000
8	Titleist Pro V1 High Numbers Personalized Gol	292	184	63.01000000000000
9	Mio ALPHA Heart Rate Monitor/Sport Watch	67	42	62.69000000000000
10	Under Armour Men's Tech II T-Shirt	280	175	62.50000000000000

Figure S15: SQL Server output screenshot 15.

SQL Query 16: Order size impact on profitability and delivery performance.

```

267
268      -- 16. How does order size impact profitability and delivery performance?
269      -- Objective:
270      -- To analyze whether large orders affect profit margins and shipping efficiency.
271      -- Business Impact:
272      -- Supports better resource allocation, shipping optimization, and bulk order management.

```

100 % ☒ No issues found

Results Messages

	order_size	total_orders	avg_sales	avg_profit	delay_percentage
1	Large	40718	249.58	27.06	57.25000000000000
2	Small	99130	211.66	22.67	57.29000000000000
3	Medium	40663	138.67	15.2	57.28000000000000

Figure S16: SQL Server output screenshot 16.

9. Power BI Dashboard and KPI Reporting

The final dashboard converts the cleaned dataset and SQL findings into an executive-level reporting interface. It includes KPI cards, revenue and profit trend, category contribution, regional revenue, high-risk product analysis, order size performance, and key insights.



Figure D1: Final Supply Chain Intelligence Power BI dashboard.

Dashboard Area	Purpose
KPI Cards	Shows total revenue, total profit, total orders, unique products, average delivery days, and delayed orders percentage.
Revenue & Profit Trend	Tracks monthly revenue and profit movement across the selected date range.
Revenue by Category	Shows category contribution to total revenue.
Revenue by Region	Compares regional revenue performance across Europe, LATAM, Pacific Asia, USCA, and Africa.
Top 10 High-Risk Products	Highlights products with the highest delivery delay risk.
Order Size Performance	Compares average sales and profit across Large, Small, and Medium order sizes.
Key Insights	Summarizes major findings for business users in one executive panel.

Dashboard KPI Cards

KPI	Value	Meaning
Total Revenue	\$36.78M	Total sales value generated by the supply chain dataset.
Total Profit	\$3.97M	Total profit after considering order-level profit values.
Total Orders	65.75K	Orders represented in the dashboard KPI view.
Unique Products	118	Distinct products available in the dataset.
Average Delivery Days	3.50	Average actual shipping duration.
Delayed Orders %	57.28%	Share of orders classified as delayed in dashboard reporting.

10. Results, Conclusion, and Future Scope

Key Findings

- The cleaned dataset contains 180,511 rows and 35 columns after preprocessing.
- Total revenue reached \$36.78M and total profit reached \$3.97M.
- Fan Shop generated the highest department-level sales and profit.
- Consumer customer segment generated the highest revenue and order count.
- Several products showed delay percentages above 65%, indicating product-level logistics risk.
- Large orders generated the highest average sales and profit.
- Delayed deliveries remain a key operational issue and require shipping process improvement.

Conclusion

The Supply Chain Intelligence Analytics project successfully transforms raw logistics and sales data into actionable operational intelligence. Python was used for cleaning and feature engineering, SQL Server was used for business-driven analysis, and Power BI was used to build a dashboard that supports executive decision-making. The project highlights delivery delay risks, profitable departments, regional revenue performance, customer behavior, and product-level operational bottlenecks.

Future Scope

- Integrate live shipment tracking data for real-time logistics monitoring.
- Add machine learning models to predict late deliveries before dispatch.
- Build supplier-level risk scoring to identify recurring fulfillment issues.
- Deploy the Power BI dashboard to Power BI Service with scheduled refresh.
- Add AI assistant support for natural language question answering over KPIs and insights.
- Create alert rules for high delay percentage, negative profit, and rising shipping time.